

# Distributed Compute & Training with Ray

A self-contained ~3-hour lecture + hands-on session: Ray Core (Tasks & Actors) • Ray Train • Data vs. Model Parallelism • Fault Tolerance

---

# Agenda — ~3 Hours

| Time        | Block                                                                         | Format              |
|-------------|-------------------------------------------------------------------------------|---------------------|
| 0:00 – 0:10 | Recap L1 + motivate: why Spark's DataFrame model doesn't fit arbitrary Python | Lecture             |
| 0:10 – 0:40 | Ray Core: tasks vs. actors vs. the object store                               | Lecture + live demo |
| 0:40 – 1:05 | Applied Task example: distributed feature engineering                         | Lecture + code      |
| 1:05 – 1:15 | Break                                                                         | —                   |
| 1:15 – 1:45 | Applied Actor example: a live experiment tracker                              | Lecture + code      |
| 1:45 – 2:15 | Ray Train: data-parallel training on a real dataset                           | Lecture + code      |
| 2:15 – 2:35 | Case studies: training an LLM across a multi-node GPU cluster; spot instances | Discussion          |
| 2:35 – 3:00 | Hands-on Exercise: tasks, an actor, and Ray Train (CPU-only)                  | Lab                 |

## PREREQUISITES

# Before You Start This Session

- Python 3.9+ with Ray installed: `pip install "ray[train]" torch`
- Confirm `import ray; ray.init()` succeeds and `ray.shutdown()` cleans up, on your machine.
- Everything today is CPU-only by design — no GPU is required or used anywhere, including in Ray Train.
- Completion of Lecture 1 (Apache Spark) is assumed — we contrast Ray's model against Spark's throughout.
- This deck is self-contained: every concept is followed by a runnable code snippet, and hands-on exercises (amber slides) reference the companion Jupyter notebook.

00

## Why Ray, When We Already Have Spark?

### What this section covers

- What Spark is (and isn't) good at
- The gap: arbitrary distributed Python
- Where Ray fits in the AIOps stack

## Spark's Sweet Spot vs. Ray's Sweet Spot

### Apache Spark

- Large, mostly-tabular ETL and feature engineering.
- A mature, SQL-like DataFrame API.
- Optimized for batch transformations over structured data.

### Ray

- Arbitrary Python: custom training loops, simulations, hyperparameter sweeps.
- Multi-GPU / multi-node model training.
- General-purpose distributed computing — minimal code changes from ordinary Python.

## Ray Picks Up Where DataFrames Stop Being the Right Model

- Reinforcement learning simulations don't map onto rows and columns.
- Order-dependent, stateful logic (a rolling score with custom reset rules) doesn't vectorize cleanly into `groupBy().agg()`.
- Hyperparameter sweeps need many independent Python processes running the SAME arbitrary code with different configs.
- Ray solves this by distributing ordinary Python functions and classes directly — no need to reshape your problem into a table.

# 01

## Ray Tasks: Distributed Feature Engineering

### What this section covers

- The future/ObjectRef pattern
- A feature that doesn't vectorize in Spark
- Sequential baseline vs. Ray tasks

# Tasks, Actors, and the Object Store

1

### Task

@ray.remote on a function. Stateless.  
Calling .remote() returns immediately with a future (ObjectRef).

2

### Actor

@ray.remote on a class. Stateful, long-lived worker process — holds state across many calls.

3

### Object Store

Ray's shared-memory store for large objects, so many tasks/actors can access the same data without repeated serialization.

# Calls Return Immediately, ray.get() Resolves Them

```
import ray
ray.init()

@ray.remote
def square(x):
    return x * x

future = square.remote(5)
print(future)      # <-- an ObjectRef, NOT the number 25 yet!

result = ray.get(future) # blocks until the task finishes, resolves the value
print(result)        # 25
```

*Live demo: print the return value of .remote() BEFORE calling ray.get() — students see an ObjectRef, not a number. This is the key “aha” moment for the async/future model.*

## A Feature That Resists Spark's DataFrame Model

- Recall Module 2 Lecture 1's case study: scaling a recommendation engine's feature pipeline from 50,000 to 5 million users.
- Spark handled the large, mostly-tabular aggregations well — but some feature logic is inherently SEQUENTIAL per user, with custom reset rules that don't express cleanly as a `groupBy().agg()`.
- Today: an “engagement momentum” score — a rolling value that decays over time but RESETS whenever a user goes quiet for too long.
- This is exactly the gap Ray fills: arbitrary Python, distributed across partitions.

# Order-Dependent, Hard to Vectorize

```
def engineer_features_plain(user_ids, seed_offset):
    results = []
    for uid in user_ids:
        gaps, clicked, lengths = generate_user_events(uid, seed_offset + uid)
        momentum = 0.0
        momentum_trace = []
        for gap, c, length in zip(gaps, clicked, lengths):
            if gap > 5.0:      # a long gap RESETS engagement
                momentum = 0.0
            momentum = 0.7 * momentum + 0.3 * (1.0 if c else 0.0) * (length / 60.0)
            momentum_trace.append(momentum)
        results.append({
            "user_id": uid, "click_rate": float(clicked.mean()),
            "final_engagement_momentum": float(momentum_trace[-1]),
            "peak_engagement_momentum": float(max(momentum_trace)),
        })
    return results
```

*Each user's momentum depends on the ORDER of their events and a conditional reset rule — exactly the kind of logic that's awkward as a DataFrame operation but trivial as plain Python.*

## @ray.remote Is the Only Change

```
@ray.remote
def engineer_features_for_partition(user_ids, seed_offset):
    return engineer_features_plain(user_ids, seed_offset) # SAME logic, unchanged

# 4,000 users split into 8 partitions, interleaved so each shard
# has similar total work
N_USERS, N_PARTITIONS = 4000, 8
user_ids = list(range(N_USERS))
partitions = [user_ids[i::N_PARTITIONS] for i in range(N_PARTITIONS)]
```

*Notice: the actual feature-engineering function is untouched. Distribution is purely additive — you don't rewrite your logic to parallelize it.*

# Measuring the Real Difference

```
# Sequential: plain Python, no Ray at all
t0 = time.time()
for p in partitions:
    engineer_features_plain(p, seed_offset=1000)
sequential_time = time.time() - t0

# Ray tasks: launch all 8 partitions concurrently
t0 = time.time()
futures = [engineer_features_for_partition.remote(p, seed_offset=1000) for p in partitions]
all_results = ray.get(futures) # blocks until every partition finishes
parallel_time = time.time() - t0

print(f"Speedup: {sequential_time / parallel_time:.1f}x")
```

*On a normal 4-8 core laptop, expect speedup to approach N\_PARTITIONS. On a constrained single-core machine, Ray's own scheduling overhead can make parallel SLOWER than sequential — a real hardware ceiling, not a bug (see the notebook's discussion prompt).*

# 02

## Ray Actors: A Live Experiment Tracker

### What this section covers

- Why a stateless task can't solve this
- Building a shared leaderboard
- Reading state back from the actor

# Six Parallel Training Runs, One Shared Leaderboard

- Now launch SIX independent model-training tasks (three model types, two configs each) in parallel.
- As they complete, we want to know which one is winning — a running leaderboard.
- A stateless `@ray.remote` TASK can't do this: each task runs in its own isolated process with no memory of what any other task found.
- We need something that PERSISTS state across many calls from many different tasks — exactly what a Ray ACTOR is for.

## 2.2 DEFINE THE ACTOR

# One Instance, Shared by Every Task

```
@ray.remote
class ExperimentTracker:
    """A stateful coordinator that many parallel training tasks report results to.
    This is exactly the shared-state problem a stateless task cannot solve alone."""
    def __init__(self):
        self.results = []
        self.best = None

    def report(self, run_name, model_type, params, accuracy):
        entry = {"run_name": run_name, "model_type": model_type,
                "params": params, "accuracy": accuracy}
        self.results.append(entry)
        if self.best is None or accuracy > self.best["accuracy"]:
            self.best = entry
        return len(self.results)

    def get_leaderboard(self, top_k=6):
        return sorted(self.results, key=lambda r: -r["accuracy"][:top_k])

tracker = ExperimentTracker.remote() # ONE actor instance, reused by every task
```

Create the actor *ONCE* outside the loop. A common mistake is calling `ExperimentTracker.remote()` inside a loop, which creates a *NEW* actor (and fresh, empty state) every time.

# A Remote Method Call on the Actor Handle

```
@ray.remote
def train_and_report(tracker_handle, run_name, model_type, params,
                    X_train, y_train, X_test, y_test):
    model = build_model(model_type, params) # RF / LogisticRegression / DecisionTree
    model.fit(X_train, y_train)
    acc = accuracy_score(y_test, model.predict(X_test))

    # A REMOTE METHOD CALL on the actor handle -- this is what lets many
    # independent tasks all update the SAME shared state safely.
    count_so_far = ray.get(tracker_handle.report.remote(run_name, model_type, params, acc))
    return run_name, acc, count_so_far

configs = [
    ("rf-shallow", "random_forest", {"n_estimators": 50, "max_depth": 4}),
    ("rf-deep", "random_forest", {"n_estimators": 50, "max_depth": 12}),
    # ... 4 more configs across 3 model types ...
]
futures = [train_and_report.remote(tracker, name, mt, p, ...) for name, mt, p in configs]
```

*Six independent tasks, one shared actor — every task's result lands in the SAME leaderboard, in whatever order the tasks happen to finish.*

# The Actor Remembers Everything

```
print("=== Leaderboard (via the actor) ===")
for entry in ray.get(tracker.get_leaderboard.remote()):
    print(f" {entry['run_name']:20s} {entry['accuracy']:.4f} {entry['params']}")

# Example output:
# rf-deep          0.8475 {'n_estimators': 50, 'max_depth': 12}
# rf-shallow       0.7975 {'n_estimators': 50, 'max_depth': 4}
# tree-deep        0.7800 {'max_depth': 15}
# logreg-l2-strong 0.7125 {'C': 0.1}

assert ray.get(tracker.count.remote()) == len(configs) # every run WAS tracked
```

*Discussion: what would happen if each call to `train_and_report.remote()` created its OWN new tracker internally, instead of sharing one? (Each task would see only its own result — the whole point of the actor would be lost.)*

# 03

## Ray Train: Data-Parallel Training

### What this section covers

- A real, learnable dataset — not random noise
- Reusing the actor pattern to monitor training
- Data parallelism vs. model parallelism
- Fault tolerance & checkpointing

## Why Not Just `torch.randn(...)`?

- A toy dataset of pure random numbers has NOTHING to learn — loss won't meaningfully fall, accuracy won't meaningfully rise.
- Today we generate a dataset with genuine class structure (scikit-learn's `make_classification`), so a falling loss reflects REAL learning, not noise.
- We'll split it ACROSS two CPU workers (data parallelism) and confirm each worker's model actually improves on its own shard.

# Data Parallelism vs. Model Parallelism

| Strategy          | What's Split                                               | When to Use                                                                        |
|-------------------|------------------------------------------------------------|------------------------------------------------------------------------------------|
| Data Parallelism  | The DATASET; each worker holds a FULL copy of the model    | The model fits on one device, but training is slow because the dataset is huge     |
| Model Parallelism | The MODEL itself, across devices (layers or tensor shards) | The model itself is too large to fit in a single device's memory (e.g. large LLMs) |

## Data Parallelism Has a Precondition

- Data parallelism **REQUIRES** the full model (parameters + optimizer state + activations) to fit in a single device's memory.
- When it doesn't fit, no amount of data parallelism helps — model parallelism becomes mandatory, not optional.
- Ray Train provides a unified API for both, integrating with PyTorch's Distributed Data Parallel (DDP) and Fully Sharded Data Parallel (FSDP) without hand-rolled multiprocessing or NCCL setup.

### 3.4 REUSING THE ACTOR PATTERN

## A Live Training Monitor

```
@ray.remote
class TrainingMonitor:
    """Same pattern as Section 2's ExperimentTracker, now coordinating live
    progress from data-parallel Ray Train workers."""
    def __init__(self):
        self.history = []

    def report_epoch(self, world_rank, epoch, train_loss, val_accuracy, shard_size):
        self.history.append({"world_rank": world_rank, "epoch": epoch,
                              "train_loss": train_loss, "val_accuracy": val_accuracy, "shard_size": shard_size})

    def get_history(self):
        return self.history

monitor = TrainingMonitor.remote()
```

*Why an actor here, and not Ray Train's own built-in metrics API? It sidesteps version-to-version API differences in Ray Train's Result object, AND it's a satisfying callback to Section 2 — the SAME shared-state pattern, applied to a new problem.*

## Data Sharding by world\_rank

```
def train_loop_per_worker(config):
    from ray.train import get_context
    ctx = get_context()
    world_size = ctx.get_world_size() # how many workers total
    world_rank = ctx.get_world_rank() # THIS worker's index

    # DATA PARALLELISM: each worker trains on a DIFFERENT slice of the data.
    X_all, y_all = config["X_train"], config["y_train"]
    shard_idx = np.arange(world_rank, len(X_all), world_size)
    X_shard = torch.tensor(X_all[shard_idx])
    y_shard = torch.tensor(y_all[shard_idx]).unsqueeze(1)

    model = build_model()
    for epoch in range(config["epochs"]):
        # ... forward, loss, backward, optimizer.step() ...
        ray.get(config["monitor"].report_epoch.remote(
            world_rank, epoch, loss.item(), val_acc, len(shard_idx)))
```

*world\_rank=0 gets rows 0, world\_size, 2\*world\_size, ...; world\_rank=1 gets rows 1, world\_size+1, ... — a simple, explicit way to see data parallelism in action.*

### 3.6 LAUNCH THE TRAINING RUN

## CPU-Only, Lightweight Workers

```
from ray.train.torch import TorchTrainer
from ray.train import ScalingConfig

trainer = TorchTrainer(
    train_loop_per_worker,
    train_loop_config={"lr": 1e-2, "epochs": 15, "monitor": monitor,
                      "X_train": X_tr, "y_train": y_tr, ...},
    scaling_config=ScalingConfig(
        num_workers=2, use_gpu=False,
        resources_per_worker={"CPU": 0.5}, # lightweight workers
    ),
)
result = trainer.fit()
```

*resources\_per\_worker={"CPU": 0.5} deliberately keeps each worker lightweight — this model/dataset is small enough that half a CPU per worker is plenty, and it means this exact code runs even on a 1-2 core machine, not just an 8-core workstation.*

### 3.7 CONFIRM REAL LEARNING HAPPENED

## Reading History Back From the Actor

```
history = ray.get(monitor.get_history.remote())
df = pd.DataFrame(history).sort_values(["world_rank", "epoch"])
print(df.to_string(index=False))

# world_rank epoch train_loss val_accuracy
#      0      0  0.688      0.610
#      0     14  0.281      0.895 <- real improvement
#      1      0  0.706      0.516
#      1     14  0.306      0.879 <- real improvement

first_loss = df.groupby("world_rank")["train_loss"].first()
last_loss = df.groupby("world_rank")["train_loss"].last()
assert (last_loss < first_loss).all() # loss DECREASED on every worker
```

*Both workers trained on DIFFERENT halves of the data yet reached similar accuracy — evidence the learnable structure in the dataset, not memorization of one shard, is what's being picked up.*

## Why It Matters More at Scale

- A training job running across 32 GPUs for 3 days has a meaningfully higher chance that AT LEAST ONE node fails mid-run than a single-machine job.
- Ray Train supports automatic checkpointing and worker recovery: if a worker dies, Ray restarts it and resumes from the last checkpoint, rather than restarting the entire job.
- Connects back to Module 1: the checkpoint itself should be logged with MLflow, so a resumed run remains fully traceable.

# 04

## Case Studies

### What this section covers

- Training an LLM across a multi-node GPU cluster
- Cost-optimization with spot instances

# Fine-Tuning an LLM Across 8 Nodes × 4 GPUs

1

### 1. Constraint

Model's params + optimizer state + activations exceed one GPU's VRAM.

2

### 2. Shard the Model

Introduce model parallelism — tensor or pipeline parallel across GPUs.

3

### 3. Hybrid Strategy

Model parallelism WITHIN a node (fast NVLink); data parallelism ACROSS nodes.

4

### 4. Checkpoint Tradeoff

Frequent checkpoints waste I/O; infrequent ones risk losing compute on failure.

# Cost-Optimization of GPU Clusters Using Spot Instances

- Spot/preemptible instances cost a fraction of on-demand GPU pricing, but can be reclaimed by the cloud provider with little warning.
- Ray's fault-tolerant task/actor model and Ray Train's checkpoint-and-resume capability make spot instances economically viable.
- A job can lose a spot-instance worker mid-training and simply resume on a new one, rather than losing the entire run.
- Discussion: today's feature-engineering tasks (Section 1) and training run (Section 3) are both natural spot-instance candidates — why? (Independent/checkpointed work, cheap to retry.)

## Ray Tasks, an Actor & Ray Train (CPU-Only)

1. Open the companion notebook: `Lecture2_Ray_Distributed_Exercise.ipynb`.
2. Section 1: engineer features for 4,000 users across 8 Ray tasks; compare sequential vs. parallel timing.
3. Section 2: train 6 model configurations in parallel; build a live leaderboard via a shared `ExperimentTracker` actor.
4. Section 3: run data-parallel Ray Train (2 CPU workers, `resources_per_worker={"CPU": 0.5}`) on a real, learnable dataset.
5. Confirm training loss DECREASED and validation accuracy ROSE on every worker — the notebook asserts this automatically.
6. Answer both discussion prompts: the actor-mistake scenario (Section 2) and synchronized data-parallel training (Section 3).

**Deliverable:** 8-task feature engineering with a timing comparison, a correct actor-based leaderboard across 6 runs, and a Ray Train run with verified decreasing loss / rising accuracy on 2+ workers.

# Ray Cheat Sheet

| Task                                     | Code                                                                  |
|------------------------------------------|-----------------------------------------------------------------------|
| Initialize Ray                           | <code>ray.init()</code>                                               |
| Define / call a task                     | <code>@ray.remote def f(x): ... then f.remote(x)</code>               |
| Resolve a future                         | <code>ray.get(future)</code>                                          |
| Define / create an actor                 | <code>@ray.remote class C: ... then handle = C.remote()</code>        |
| Call an actor method                     | <code>ray.get(handle.method.remote(...))</code>                       |
| Distributed training                     | <code>TorchTrainer(loop_fn, scaling_config=ScalingConfig(...))</code> |
| This worker's rank / world size          | <code>get_context().get_world_rank() / .get_world_size()</code>       |
| Lightweight workers (fit small hardware) | <code>ScalingConfig(..., resources_per_worker={"CPU": 0.5})</code>    |

## KEY TAKEAWAY

**Spark distributes tables. Ray distributes code.**

**Tasks (stateless) + Actors (stateful) + Ray Train (parallel training) = arbitrary Python at cluster scale**

*The same fault-tolerance model that makes spot instances economical also protects a 3-day training job from a single flaky node — and the same actor pattern that built today's leaderboard also monitored today's training run.*

*Next: Lecture 3 — Feature Stores (Feast) & Data Quality Validation (Great Expectations).*